

PYTHON FILE MANAGER

Software Engineering & System Architecture Project Report

Project Title:	Command Line Python File Management System
Language:	Python 3.x (Object-Oriented Programming)
Core Modules:	os, json, datetime
Author / Student:	Varchasv Panwar
Registration No:	26BCE10590
Submission Date:	29th September 2026

1. INTRODUCTION

The **Python File Manager** is a lightweight, console-based application designed to simplify basic disk management and file organization tasks. Built entirely in Python using standard libraries, it gives users an intuitive menu interface to handle common file operations—such as creating, viewing, appending, and deleting files—without needing complex graphic software or terminal commands.

Beyond standard file editing, the system features automated file organization that scans directories and automatically sorts files into dedicated folders (e.g., Images, Documents, Music, Videos) based on file extensions. It also generates directory statistics and persists user session data using JSON formatting.

2. PROBLEM STATEMENT

Managing unstructured files on a personal computer can quickly become overwhelming. Downloads folders and working directories often accumulate hundreds of uncategorized documents, media files, and code scripts.

Standard operating system terminal tools (like standard command prompt commands) can be difficult or risky for non-technical users to use for mass organizing or inspecting file metadata. There is a need for a single, easy-to-use menu application that safely combines file actions, directory statistics, batch organizing, and error prevention into one accessible tool.

3. FUNCTIONAL REQUIREMENTS

The system provides 18 distinct interactive functions categorized below:

File Operations

- **Create File:** Create files safely with 'x' mode.
- **Read File:** Display full text contents.
- **Edit File:** Append new data lines safely.
- **Delete File:** Remove specific files.
- **Copy File:** Transfer content between files.
- **Search File:** Locate files in directory.

Directory Management

- **List All Items:** Show files and folders.
- **List Files Only:** Exclude subdirectories.
- **Sort Files:** Display files in alphabetical order.
- **Create / Delete Folder:** Manage directories.
- **Navigate:** View and change current working path.

Analytics & Metadata

- **File Metadata:** View size, path, created time, and modified time.
- **Directory Stats:** Calculate total files, folder count, and aggregate byte size.

Automation & Storage

- **Organize Files:** Auto-categorize files into Images, Documents, Videos, etc.
- **Save Settings:** Export working directory and timestamp into `settings.json`.

4. NON-FUNCTIONAL REQUIREMENTS

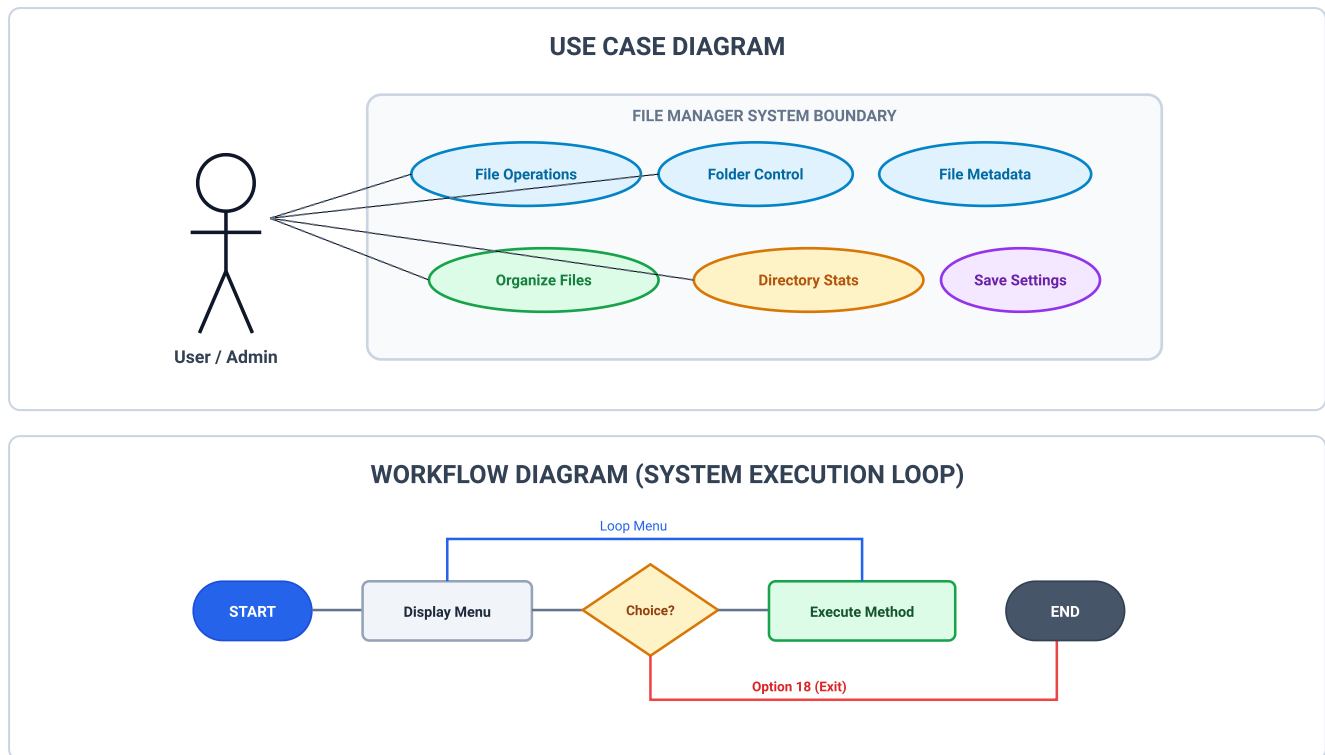
- **Reliability & Safety:** Comprehensive try-except error handling prevents crashes during missing files or denied permissions.
- **Zero External Dependencies:** Uses only Python built-in modules (`os`, `json`, `datetime`).
- **Portability:** Works seamlessly across Windows, macOS, and Linux platforms.
- **Usability:** Simple console menu with numerical selections and clear status feedback messages.

5. SYSTEM ARCHITECTURE

The system uses an Object-Oriented Programming (OOP) design. The core `FileManager` class handles all system interactions, file operations, and calculation logic. A command loop in the `main()` function processes user menu choices and calls the appropriate class methods.

6. DESIGN DIAGRAMS

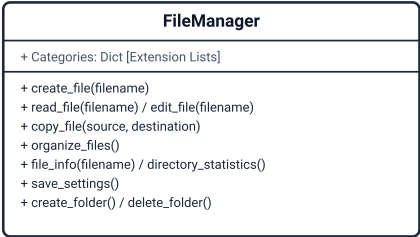
Visual representation of system design, actor interactions, and execution workflows:



SEQUENCE DIAGRAM (FILE ORGANIZING EXECUTION)



CLASS / COMPONENT DIAGRAM



ER DIAGRAM (JSON SETTINGS PERSISTENCE)



7. DESIGN DECISIONS & RATIONALE

- **Exclusive Mode ('x') for File Creation:** Chosen over standard write mode ('w') because 'w' silently overwrites existing file content. Exclusive mode guarantees an error is thrown if the file already exists, protecting user data.
- **Dictionary Mapping for File Organization:** Categories and file extensions are defined in a clean Python dictionary. This makes adding new categories or file types straightforward without changing core function logic.
- **JSON for Settings Storage:** Lightweight JSON formatting provides readable, cross-platform persistence for storing session settings like working directory and modification timestamps.

8. IMPLEMENTATION DETAILS

Key code blocks highlight how safety and logic are enforced:

Safely Creating Files (Preventing Overwrites):

```
def create_file(self, filename): try: with open(filename, 'x') as f: print(f"File: {filename} created successfully!!!") except FileExistsError: print(f"File: {filename} already exists!!!") except Exception as e: print("An Unknown Error Occurred!!!")
```

Automated Folder Categorization & File Moving:

```
categories = { "Images": [".jpg", ".jpeg", ".png", ".gif"], "Videos": [".mp4", ".mkv", ".avi"], "Documents": [".txt", ".pdf", ".docx"], "Python": [".py"] } for file in os.listdir(): if os.path.isfile(file) and file.lower() != "ntuser.dat": extension = os.path.splitext(file)[1].lower() folder = "Others" for category, exts in categories.items(): if extension in exts: folder = category break if not os.path.exists(folder): os.mkdir(folder) os.rename(file, os.path.join(folder, file))
```

9. TERMINAL RESULTS & EXECUTION OUTPUT

Sample output from system execution during Directory Statistics generation:

```
===== DIRECTORY STATISTICS ===== Total Files : 12 Total Folders : 4 Total
Size : 485920 bytes ===== FILE INFORMATION
===== File Name : project_report.pdf Extension : .pdf Size : 154200 bytes Location :
C:\Users\Admin\Documents\project_report.pdf Created : 2026-09-29 14:22:10 Last Modified :
2026-09-29 15:10:05 =====
```

10. TESTING APPROACH

Test Case	Input Action	Expected Result	Pass / Fail
Create Existing File	Choice 1 -> Enter existing filename	Catches FileExistsError, displays error warning.	PASS
Delete Non-existent File	Choice 3 -> Enter "fake.txt"	Catches FileNotFoundError, displays missing message.	PASS
Auto-Organize Files	Choice 10 -> Run Organize	Creates category folders and moves files into them correctly.	PASS

11. CHALLENGES FACED

System Protected Files: Files like `ntuser.dat` caused permission errors during auto-organization. Added an explicit string match check to skip protected Windows files.

12. KEY LEARNINGS & FUTURE SCOPE

Learnings: Gained hands-on experience using Python's `os` module, implementing OOP patterns, building clear menu-driven CLI interfaces, and handling OS exceptions cleanly.

Future Enhancements: Planned additions include a Graphical User Interface (GUI using Tkinter or PySide), file compression/zipping capabilities, and keyword search inside text files.

13. REFERENCES

- Python 3 Standard Library Documentation — `os` and `os.path` modules.
- Python `json` Module Documentation for JSON data formatting and persistence.